ACTIVIDAD DE AULA · PIPELINE DE UNA EMPRESA

Google · YouTube Analytics: de la reproducción de un vídeo al informe del creador

Sistemas de Big Data · UT1 · Pareja: Cristóbal Molina Hernández y Ángel Candamo Fernández · Octubre de 2026

Estudiamos el servicio Estadísticas de YouTube (YouTube Analytics) a partir de la arquitectura que publicaron los ingenieros de Google en 2019 (Procella) y de la documentación oficial vigente en 2026. No describimos el sistema de recomendaciones ni el cálculo de ingresos publicitarios.

Apartado 1 · Ficha del caso

1.1 ¿Qué empresa y qué servicio vamos a estudiar?

La empresa es **Google**, propietaria de **YouTube**. El servicio es **Estadísticas de YouTube** (YouTube Analytics), la sección de YouTube Studio donde cada creador consulta cómo rinden sus vídeos, y su versión para programas, la **API de YouTube Analytics**. [3, 5]

Problema que resuelve: un vídeo puede reproducirse millones de veces en muchos países y dispositivos. El creador no puede leer esas interacciones una a una; necesita unas pocas cifras fiables (visualizaciones, tiempo de visualización, suscriptores) por día, vídeo o país para decidir qué publicar y cuándo. **Destinatario:** el creador o propietario del canal y, si este lo autoriza, las aplicaciones que consultan la API. [1, 5]

1.2 ¿Qué datos entran, qué se hace con ellos y qué se obtiene?

Entran las interacciones de los espectadores con los vídeos: reproducciones, tiempo visto, Me gusta y suscripciones. Se validan, se cargan en tablas por dos caminos (tiempo real y lotes) y se agregan con consultas SQL en Procella, el motor que YouTube construyó para esto. Se obtienen informes con métricas por día, vídeo, país o dispositivo. [1, 2]

Fuente principal: el artículo «Procella: Unifying serving and analytical data at YouTube», escrito por 23 ingenieros de Google y publicado en VLDB en 2019 [1]. Lo completamos con páginas de ayuda de YouTube y la documentación de la API, consultadas el 27/10/2026 [2–6]. **Época:** la arquitectura corresponde a 2019; no podemos asegurar que en 2026 siga funcionando igual.

Introducción

Según los propios ingenieros de YouTube, la plataforma genera cada día billones de nuevos datos a partir de miles de millones de vídeos, visualizaciones y horas de reproducción. Esos datos sirven para cuatro cosas: informes para los creadores, contadores públicos como el número de visualizaciones de un vídeo, vigilancia del estado del servicio y análisis internos. [1]

Nos centramos en el primer uso: cómo una reproducción hecha por un espectador acaba convertida en una cifra del informe que el creador ve en YouTube Studio. Seguimos dos métricas concretas, **visualizaciones** y **tiempo de visualización**, porque son las que aparecen primero en la vista general del canal [5] y porque permiten un ejemplo numérico sencillo. Antes de 2019, YouTube repartía estos usos entre varios sistemas (Dremel, Mesa, Bigtable, Monarch, Vitess), con cargas de datos duplicadas e inconsistencias; Procella los unificó en un solo motor [1]. Esa es la arquitectura que describimos y la que representa la Figura 1.

Apartado 2 · Desarrollo de las preguntas

2.1 ¿De dónde salen los datos y cómo llegan al sistema?

Qué datos se necesitan. Para calcular visualizaciones y tiempo de visualización hacen falta las interacciones de los espectadores: que una reproducción ha empezado (desde el 24 de agosto de 2026 la visualización se cuenta al comenzar la reproducción, en todos los formatos [2]) y cuánto tiempo se ha visto. También hace falta saber a qué canal pertenece cada vídeo, porque cada creador solo puede consultar sus propios datos [1]; esta segunda fuente (el catálogo) es una deducción nuestra y la marcamos como propuesta en la Figura 1.

Fuente frente a mecanismo de adquisición. La *fuente* son los espectadores usando los reproductores de YouTube (web, aplicaciones móviles, televisores). El *mecanismo de adquisición* documentado tiene dos entradas a Procella [1]:

- **Tiempo real:** el *Ingestion Server* (IgS) recibe los datos, los adapta a la estructura de la tabla y los añade a un registro en disco, de forma que se pueden consultar en segundos.
- **Lotes:** procesos fuera de línea (el artículo pone como ejemplo MapReduce) generan ficheros de tabla y los registran en Procella a través del *Registration Server* (RgS).

Lo que **no** está publicado es cómo viaja el evento desde el móvil o el navegador del espectador hasta esas dos entradas; en la Figura 1 aparece como bloque gris no confirmado.

Tipo de datos. Las tablas de Procella tienen columnas con nombre y tipo declarados al crearlas, así que los datos que llegan a ellas son **estructurados** [1]. Procella también admite campos anidados y repetidos (un registro que contiene listas de subregistros), es decir, datos **semiestructurados** [1], aunque no sabemos qué campos de YouTube Analytics los usan. El propio vídeo (imagen y sonido) sería un dato **no estructurado**, pero no interviene en este pipeline: se analizan las interacciones, no el contenido.

Campos. La documentación de la API publica las métricas y dimensiones que recibe el creador (tabla 1). Los campos del evento de entrada no están publicados, así que para el ejemplo del apartado 2.2 hemos elegido los nuestros (tabla 2).

Tabla 1. Campos de salida publicados por Google en la API de YouTube Analytics [3, 4].

Campo	Significado	Unidad o formato
video	Identificador del vídeo	Texto (ID de YouTube)
day	Día al que corresponde la actividad	AAAA-MM-DD; el día empieza y acaba a las 00:00 hora del Pacífico (UTC−7 o UTC−8)
country	País asociado a la actividad	Código ISO 3166-1 de dos letras; ZZ si no se conoce
views	Visualizaciones	Número de veces
estimatedMinutesWatched	Tiempo que los usuarios han visto los vídeos	Minutos
averageViewDuration	Duración media de las reproducciones	Segundos

Tabla 2. Campos del evento de entrada elegidos por nosotros para el ejemplo (no son el esquema real de YouTube).

Campo	Significado	Unidad o formato
id_evento	Identificador de cada envío del reproductor	Texto
video	Vídeo reproducido	Texto (ID)
momento	Instante de la reproducción tal como lo envía el dispositivo	Fecha y hora ISO 8601 con su desfase horario
tiempo_visto	Cuánto se ha visto en esa reproducción	Milisegundos (web y Android) o segundos (TV), según el dispositivo: supuesto ficticio
dispositivo	Tipo de reproductor	web, Android, TV (la API tiene una dimensión parecida, deviceType [4])

2.2 ¿Qué hay que preparar antes de utilizarlos?

Lo documentado. YouTube confirma las interacciones mediante algoritmos y descarta las reproducciones de baja calidad, por ejemplo, las del mismo vídeo abierto en varias pestañas o dispositivos. Para comprobarlas puede ralentizar, detener o corregir temporalmente el recuento, y las métricas verificadas pueden tardar varios días en actualizarse [2]. Dentro de Procella, el IgS adapta cada fila a la estructura de la tabla; el RgS hace comprobaciones de coherencia al registrar ficheros; y el *Compaction Server* puede filtrar, agregar, eliminar datos antiguos o quedarse solo con el último valor mediante reglas SQL [1]. Los ficheros por lotes se particionan y ordenan para que las consultas lean lo mínimo [1].

Nuestro ejemplo (ficticio). Los registros, identificadores y cifras siguientes son inventados. Representan reproducciones del vídeo V01 del canal C01 hechas desde España. Suponemos que la segunda fila es un reenvío técnico de E101 (el móvil repitió el envío), no una segunda reproducción.

Tabla 3. Registros ficticios antes de la preparación.

id_evento	video	momento (como llega)	tiempo_visto	dispositivo
E101	V01	2026-10-05T20:00:00+02:00	240000 ms	Android
E101	V01	2026-10-05T20:00:00+02:00	240000 ms	Android (reenvío)
E102	V01	2026-10-06T05:30:00+02:00	300000 ms	web
E103	V01	2026-10-05T18:00:00Z	90 s	TV

Aplicamos tres operaciones de normalización, es decir, para hacer compatibles los datos:

- **Eliminar el reenvío:** las dos filas E101 tienen el mismo identificador y los mismos valores, así que conservamos una.
- **Unificar la fecha:** la API agrupa los días en hora del Pacífico [4]. El 5 y el 6 de octubre de 2026 rige el horario de verano del Pacífico (UTC−7). E101: 20:00 en España = 18:00 UTC = 11:00 del día 5. E102: 05:30 del día 6 en España = 03:30 UTC = **20:30 del día 5** en el Pacífico. E103: 18:00 UTC = 11:00 del día 5.
- **Unificar unidades:** pasamos el tiempo a minutos, como estimatedMinutesWatched [3]: milisegundos ÷ 60 000 y segundos ÷ 60.

Tabla 4. Los mismos registros después de la preparación.

id_evento	video	día (hora del Pacífico)	minutos vistos
E101	V01	2026-10-05	4,0
E102	V01	2026-10-05	5,0
E103	V01	2026-10-05	1,5

Resultado para el 5 de octubre: **3 visualizaciones y 10,5 minutos**. Si no se preparan los datos, la salida parece válida pero es falsa:

- Sin eliminar el reenvío: 4 visualizaciones y 14,5 minutos.

- Sin convertir la zona horaria: E102 se iría al día 6 (la fecha que vio el espectador en España), y el día 5 mostraría 2 visualizaciones y 5,5 minutos.
- Sin unificar unidades: sumar los números en bruto da 780 090, una cifra sin sentido porque mezcla milisegundos y segundos.

Es una simplificación didáctica; no afirmamos que reproduzca el algoritmo de Google. Una regla por identificador elimina reenvíos técnicos, pero **no** debe borrar dos reproducciones distintas del mismo vídeo solo porque coincidan el vídeo y el espectador. Que YouTube considere de baja calidad algunas repeticiones es una regla suya, documentada en [2], no de nuestro ejemplo.

2.3 ¿Qué información se guarda y para qué?

- **Datos originales recientes.** El IgS escribe cada fila en un registro (*write-ahead log*) en Colossus, el sistema de ficheros distribuido de Google, y la mantiene a la vez en memoria. Así se puede consultar en segundos, aunque todavía no esté consolidada; ese registro sirve también para recuperarse si un servidor falla. [1]
- **Datos preparados.** Todos los datos duraderos se guardan en Colossus, separados de los servidores que calculan, que se ejecutan en Borg, el planificador de contenedores de Google. Esta separación permite aumentar los servidores de cálculo sin mover los datos. Las tablas usan **Artus**, un formato columnar propio pensado tanto para leer muchas filas como para buscar una fila concreta muy rápido; también puede leer ficheros Capacitor, el formato de BigQuery. [1]
- **Metadatos.** Los esquemas, la relación entre tablas y ficheros, las estadísticas y los mínimos y máximos de cada fichero se guardan en Bigtable y Spanner. Sirven para descartar ficheros antes de leerlos. [1]
- **Resultados.** El artículo no describe un almacén separado para los informes finales: los resultados se calculan al consultar. Procella admite tablas virtuales que eligen versiones preagregadas (vistas materializadas) para acelerar consultas repetidas [1], y los servidores guardan en caché los datos usados con frecuencia. No hemos podido confirmar qué informes concretos de YouTube Analytics usan esas tablas preagregadas.

Por tanto, no hay un almacén distinto para cada tipo de dato: Colossus guarda tanto el registro reciente como las tablas preparadas, en formatos y con funciones diferentes.

2.4 ¿Cómo se obtiene el resultado?

Cuando el creador abre un informe, se lanza una consulta SQL. El **Root Server** la recibe, consulta los metadatos para descartar los ficheros que no contienen datos de ese canal o de esas fechas y reparte el trabajo. Los **Data Servers** leen las columnas necesarias, filtran, agrupan y suman, y devuelven el resultado [1]. Como cada creador solo ve sus datos, cada consulta toca una porción pequeña de un conjunto enorme [1]. Para nuestro ejemplo, la operación sería (SQL escrita por nosotros, no la de YouTube):

```
SELECT day, SUM(views), SUM(minutes)
FROM interacciones_preparadas
WHERE channel = 'C01' AND day = '2026-10-05'
GROUP BY day;    -- resultado: 2026-10-05 | 3 | 10,5
```

Lotes y tiempo real combinados. Procella implementa una *arquitectura lambda*: los datos en tiempo real están disponibles antes, los de lotes son más exactos y completos pero llegan más tarde, y a intervalos regulares los de lotes sustituyen a los de tiempo real. Al consultar, se unen con UNION ALL las tablas que cubren distintos tramos de tiempo [1]. De las más de 1500 millones de consultas diarias de la instancia de YouTube Analytics, más de 700 millones trabajaban sobre datos en tiempo real; la mediana de respuesta era de 25 ms [1].

Por qué encaja con el servicio. El creador quiere ver cómo funciona un vídeo justo después de publicarlo: el informe en tiempo real compara los últimos 60 minutos y las últimas 48 horas, y YouTube avisa de que es una *estimación* [2, 5]. Pero también necesita cifras definitivas, y la verificación de visualizaciones puede tardar días [2]. Relacionar esa verificación con el camino por lotes es una interpretación nuestra: las fuentes describen ambos hechos, pero no cómo se conectan.

2.5 ¿Qué recibe el destinatario y cómo lo utiliza?

En YouTube Studio > Estadísticas, el creador ve una vista general con visualizaciones, tiempo de visualización en horas, suscriptores e ingresos estimados; el informe en tiempo real; y los vídeos principales del periodo [5]. Con la API, otras aplicaciones obtienen las mismas métricas por día, país o dispositivo [3, 4]. Los millones de interacciones sueltas del principio se convierten en unas pocas cifras por día que le ayudan a decidir, por ejemplo, a qué hora publicar o qué tipo de vídeo funciona mejor. Si un desglose tiene poco tráfico, YouTube oculta esa fila por privacidad, de modo que el total puede no coincidir con la suma de las filas [6].

Características de Big Data. Destaca el **volumen**: cada fuente añade a menudo cientos de miles de millones de filas al día, y la instancia de YouTube Analytics recorría más de 80 000 billones de filas diarias sobre decenas de petabytes [1]. También la **velocidad**: los contadores públicos deben aplicar millones de actualizaciones en tiempo real mientras atienden millones de consultas por segundo [1]. A ellas se suma la **veracidad**: hay que separar las reproducciones reales de las automáticas o repetidas antes de dar una cifra por buena [2].

Apartado 3 · Esquema del pipeline

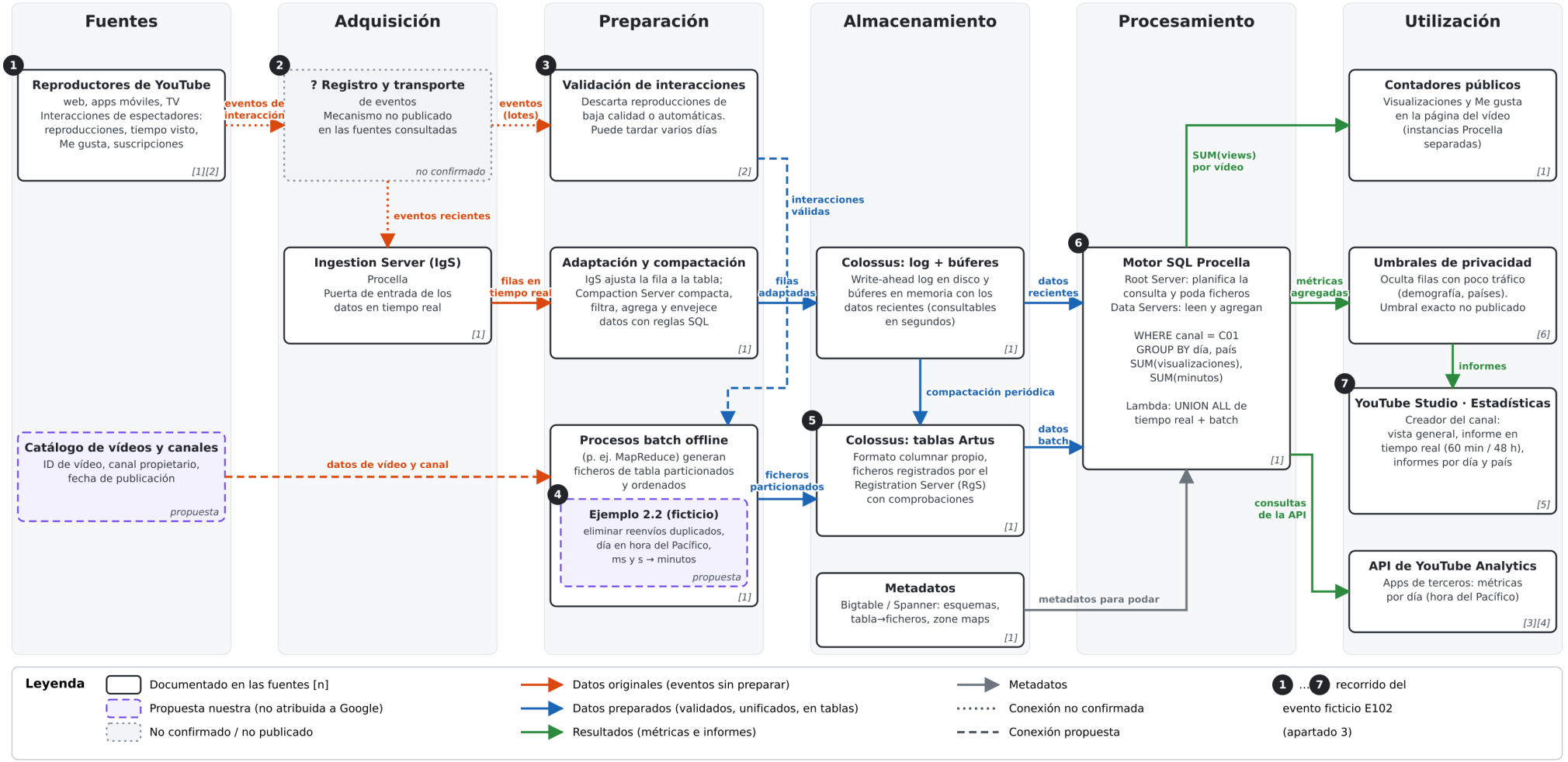


Figura 1. Pipeline de Estadísticas de YouTube (YouTube Analytics), elaborado por nosotros. Los componentes de Procella (IgS, RgS, Compaction Server, Colossus, Artus, metadatos y motor SQL) proceden del artículo de 2019 [1]; la validación de interacciones, de [2]; los formatos de salida, de [3, 4]; y el uso por el creador y los umbrales de privacidad, de [5, 6]. Las cajas moradas discontinuas son propuestas nuestras y las grises punteadas, partes que no hemos podido confirmar. Los números 1 a 7 siguen el evento ficticio E102.

Cómo leer la figura. Las columnas siguen las funciones del pipeline. Las flechas naranjas transportan datos originales, las azules datos preparados y las verdes resultados. La rama superior lleva datos validados por lotes; la central, datos en tiempo real que se pueden consultar en segundos y que el Compaction Server consolida periódicamente en tablas Artus. Ambas ramas se unen en el motor SQL, que también alimenta los contadores públicos de la página del vídeo mediante instancias separadas [1].

Seguimos un dato: el evento ficticio E102 (Figura 1, números 1 a 7).

1. Un espectador en España ve 5 minutos de V01 en la web a las 05:30 del 6 de octubre. El reproductor genera el evento con 300 000 ms.
2. El evento viaja por el sistema de registro de Google; ese tramo no está publicado y la flecha es punteada.
3. La validación comprueba que es una reproducción real y no, por ejemplo, el mismo vídeo abierto en varias pestañas [2].
4. En la preparación por lotes (nuestra propuesta del 2.2), su fecha pasa a 2026-10-05 en hora del Pacífico y su duración a 5,0 minutos.
5. Se guarda en una tabla Artus de Colossus, registrada por el RgS [1].
6. Procella lo suma con E101 y E103: 3 visualizaciones y 10,5 minutos para el 5 de octubre [1].
7. El creador ve esa cifra en la barra del 5 de octubre en YouTube Studio [5]. Antes de que llegue el lote, una versión estimada de esa reproducción pudo aparecer en el informe en tiempo real por la rama central.

Si E102 hubiera conservado la fecha española, el paso 4 la habría llevado al día 6. Ese es el cambio que hace visible la figura.

Apartado 4 · Qué podría fallar

4.1 ¿Qué pasaría si llegaran datos incorrectos?

Problema elegido: unidades mezcladas. En nuestro ejemplo, unos dispositivos envían milisegundos y otros segundos. Si en la preparación se tratasen los milisegundos como si fueran segundos, E101 sumaría 240 000 s (4000 min) y E102, 300 000 s (5000 min). El total del 5 de octubre pasaría de 10,5 a 9001,5 minutos, y la duración media de las visualizaciones se dispararía. Nada fallaría de forma visible: el creador vería una cifra posible y podría sacar conclusiones equivocadas sobre qué vídeo retiene a la audiencia.

Dónde detectarlo (propuesta). En la preparación, antes de registrar el fichero, igual que el RgS hace comprobaciones de coherencia [1]. Proponemos dos comprobaciones: que cada dispositivo declare la unidad en la que envía el tiempo, y que ninguna reproducción supere la duración del vídeo. Si V01 dura 12 minutos (dato ficticio), 4000 minutos son imposibles y el lote debería apartarse para revisarlo. Una tercera alarma sencilla sería comparar la duración media del día con la de días anteriores: un salto de cientos de veces indica un error de unidades más que un éxito del vídeo.

4.2 ¿Qué parte no hemos podido confirmar?

- Cómo se capturan y transportan los eventos desde los reproductores hasta Procella. Las fuentes no lo describen.
- El algoritmo exacto de validación y en qué punto del pipeline se aplica. YouTube no lo publica, lo que tiene sentido para que no se pueda burlar [2].
- El esquema real de las tablas de YouTube Analytics: nuestros campos de entrada y nuestras re-

glas de preparación son inventados.

- Si los informes finales se guardan preagregados o se calculan siempre al consultar.
- Si en 2026 YouTube Analytics sigue funcionando sobre Procella con la arquitectura y las cifras de 2019 [1].
- Los umbrales exactos a partir de los cuales se ocultan datos por privacidad, que YouTube no publica [6].
- El efecto del cambio de agosto de 2026 en la forma de contar visualizaciones [2] sobre la comparación con datos anteriores.

4.3 ¿Se utilizan datos personales o sensibles?

Sí. Cada interacción está ligada a un espectador. La API ofrece edad y sexo de los usuarios con sesión iniciada, incluidos usuarios que YouTube estima menores de 18 años, y la ubicación (país y ciudad estimada) [4]. Además, el historial de vídeos vistos puede revelar información sensible, como salud, ideología o religión; esta última es una observación nuestra.

Medidas documentadas: el creador solo recibe datos agregados; las filas con poco tráfico se ocultan; y los vídeos privados u ocultos no muestran datos demográficos [6]. **Nuestra propuesta:** sustituir el identificador de la cuenta por un seudónimo en cuanto el evento entra en el pipeline, guardando la correspondencia aparte y con acceso restringido, y que las tablas que alimentan los informes contengan solo agregados por día, vídeo y país, nunca filas por espectador. Nuestros ejemplos no contienen datos personales reales: los eventos no llevan ningún campo de usuario.

Conclusiones

El pipeline convierte un flujo enorme de interacciones sueltas en unas pocas cifras por canal y día que el creador puede consultar en milisegundos. Lo consigue separando dos caminos (rápido y estimado, lento y verificado) y uniéndolos en un único motor SQL que lee datos preparados y bien organizados en Colossus [1].

La etapa que nos parece más importante es la **preparación y validación**. Nuestro ejemplo muestra que, con los mismos tres eventos ficticios, el día 5 puede salir con 10,5 minutos (correcto), 14,5 (reenvío sin eliminar), 5,5 (zona horaria equivocada) o 9001,5 (unidades mezcladas), y las tres cifras erróneas parecen creíbles. La propia YouTube prefiere congelar o corregir un contador antes que mostrar visualizaciones no verificadas [2]. Además, la rapidez de Procella solo es útil si lo que suma es correcto.

Referencias

[1] B. Chattopadhyay, P. Dutta, W. Liu, O. Tinn, A. McCormick, A. Mokashi, P. Harvey, H. Gonzalez, D. Lomax, S. Mittal et al. (Google LLC). «Procella: Unifying serving and analytical data at YouTube». *Proceedings of the VLDB Endowment*, 12(12), pp. 2022-2034. 2019. http://www.vldb.org/pvldb/vol12/p2022-chattopadhyay.pdf

[2] Ayuda de YouTube (Google). «Cómo se contabilizan las métricas de interacción». Sin fecha de publicación; incluye el cambio del 24/08/2026. Consultado el 27/10/2026. https://support.google.com/youtube/answer/2991785?hl=es

[3] Google for Developers. «YouTube Analytics API: Metrics». Sin fecha de publicación; consultado el 27/10/2026. https://developers.google.com/youtube/analytics/metrics

[4] Google for Developers. «YouTube Analytics API: Dimensions». Sin fecha de publicación; consultado el 27/10/2026. https://developers.google.com/youtube/analytics/dimensions

[5] Ayuda de YouTube (Google). «Obtener una vista general del rendimiento del canal». Sin fecha de publicación; consultado el 27/10/2026. https://support.google.com/youtube/answer/6096647?hl=es

[6] Ayuda de YouTube (Google). «Interpretar los datos limitados en YouTube Analytics». Sin fecha de publicación; consultado el 27/10/2026. https://support.google.com/youtube/answer/9101241?hl=es